

```

/*
=====

=====
* PARKBNB BARRIER CONTROLLER
*
=====

=====
* ESP32 + MPU6050 + BTS7960 + Buzzer + LEDs + Web Server + Firebase
*
* POSITION CALCULATION
*
-----
----
* Position is calculated using BOTH the X and Z accelerometer axes:
*
*     positionAngle = atan2(filteredAx, filteredAz) * 180 / PI
*
* Based on actual ParkBnB MPU measurements:
*
*     Physical 0°    -> approximately -73°
*     Physical 45°   -> approximately -31°
*     Physical 90°   -> approximately  +6°
*     Physical 120°  -> approximately +50°
*     Physical 180°  -> approximately +90°
*
* This lets the firmware distinguish:
*     - BELOW physical 90°
*     - ABOVE physical 90°
* and therefore select the correct motor direction to return to 90°.
*
=====

=====
*/

#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <math.h>

// Firebase-related includes
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include <ArduinoJson.h>

```

```

/*
=====
=====
* SECTION 1: WI-FI CREDENTIALS
*
=====
=====
*/
const char* ssid      = "D-Link_DIR-615";
const char* password = "123456789";

/*
=====
=====
* SECTION 2: FIREBASE CONFIGURATION
*
=====
=====
* NOTE: Do NOT add a trailing "/" to DATABASE_URL.
*/
#define DATABASE_URL
"https://parkfnb-a382e-default-rtdb.asia-southeast1.firebaseio.com"
#define BARRIER_ID  "BARRIER_001"

// Firebase REST endpoint URLs (built at runtime in
initializeFirebasePaths())
String firebaseDeviceURL;
String firebaseLiveDataURL;
String firebaseCommandURL;

// Firebase live-data push timing
unsigned long lastFirebaseLiveUpdate = 0;
const unsigned long FIREBASE_LIVE_UPDATE_INTERVAL = 5000;    // ms

// Firebase command polling timing
unsigned long lastFirebaseCommandCheck = 0;
const unsigned long FIREBASE_COMMAND_CHECK_INTERVAL = 2000;  // ms

```

```

/*
=====

=====
* SECTION 3: MPU6050 (ACCELEROMETER/GYRO)
*
=====

=====
* /
#define MPU_ADDR      0x68
#define MPU_PWR_MGMT_1 0x6B
#define MPU_ACCEL_XOUT 0x3B

/*
=====

=====
* SECTION 4: BTS7960 MOTOR DRIVER PINS
*
=====

=====
* /
#define RPWM_PIN 18
#define LPWM_PIN 19
#define REN_PIN  25
#define LEN_PIN  26

/*
=====

=====
* SECTION 5: BUZZER
*
=====

=====
* /
#define BUZZER_PIN      27
#define BUZZER_DURATION 5000UL // ms

/*
=====

=====
* SECTION 6: STATUS LEDs

```

```

*
=====

=====
* /
#define LED_NINETY_PIN 32
#define LED_ZERO_PIN 33

/*
=====

=====
* SECTION 7: MOTOR SETTINGS
*
=====

=====
* /
#define MOTOR_SPEED 80
#define PWM_FREQ 1000
#define PWM_RESOLUTION 8

/*
=====

=====
* SECTION 8: POSITION CALIBRATION
*
=====

=====
* These reference values are based on real MPU readings taken on the
* physical ParkBnB rig:
*
* Physical 0 degrees:
* AX  $\approx$  -0.948, AZ  $\approx$  0.295 -> atan2(AX, AZ)  $\approx$  -72.7°
*
* Physical 90 degrees:
* AX  $\approx$  0.117, AZ  $\approx$  1.035 -> atan2(AX, AZ)  $\approx$  6.4°
* /
#define ZERO_ANGLE_REFERENCE -72.7f
#define NINETY_ANGLE_REFERENCE 6.4f

/* ---- 90° tolerance window
-----
* Approximately +/- 4 degrees around the calibrated 90° position.

```

```

    * This prevents small vibrations from falsely triggering the security
    logic.
    */
#define NINETY_ANGLE_TOLERANCE 4.0f
#define NINETY_LOWER_LIMIT (NINETY_ANGLE_REFERENCE -
NINETY_ANGLE_TOLERANCE)
#define NINETY_UPPER_LIMIT (NINETY_ANGLE_REFERENCE +
NINETY_ANGLE_TOLERANCE)

/* ---- 0° tolerance window
-----

    */
#define ZERO_ANGLE_TOLERANCE 4.0f
#define ZERO_UPPER_LIMIT (ZERO_ANGLE_REFERENCE + ZERO_ANGLE_TOLERANCE)

/* ---- Legacy AZ threshold
-----

    * Kept for LED compatibility with the original system.
    */
#define ZERO_THRESHOLD 0.35f

/*
=====
=====
    * SECTION 9: STALL DETECTION
    *
    =====
=====
    */
#define STALL_CHECK_INTERVAL 100UL // ms between stall checks
#define MIN_MOVEMENT 0.20f // minimum angle change
considered "moving"
#define MAX_NO_MOVEMENT_COUNT 3 // consecutive stalled checks
before alarm

/*
=====
=====
    * SECTION 10: MOTOR TIMEOUT

```

```

*
=====

=====
*/
#define MOTOR_TIMEOUT 15000UL // ms

/*
=====

=====
* SECTION 11: OBSTACLE RETRY
*
=====

=====
*/
#define OBSTACLE_RETRY_DELAY 100UL // ms

/*
=====

=====
* GLOBAL OBJECTS
*
=====

=====
*/
WebServer server(80);

/*
=====

=====
* MOTOR STATE MACHINE
*
=====

=====
*/
enum MotorState
{
    STOPPED,
    MOVING_TO_ZERO,
    MOVING_TO_NINETY,
    MOVING_DOWN_FROM_OVER_NINETY
}

```

```

};

MotorState motorState      = STOPPED;
MotorState commandedTarget = STOPPED;

/*
=====
=====
* MPU VARIABLES
*
=====
=====
*/
float ax = 0.0f;
float az = 0.0f;

float filteredAx = 0.0f;
float filteredAz = 0.0f;

// Calculated barrier position angle (the key position variable).
float positionAngle = 0.0f;

/*
=====
=====
* STALL DETECTION VARIABLES
*
=====
=====
*/
float lastStallCheckAngle = 0.0f;
unsigned long lastStallCheckTime = 0;
int noMovementCount = 0;

/*
=====
=====
* MOTOR TIMER

```

```

*
=====

=====
*/
unsigned long motorStartTime = 0;

/*
=====

=====
* BUZZER STATE
*
=====

=====
*/
bool buzzerActive = false;
unsigned long buzzerStartTime = 0;

/*
=====

=====
* OBSTACLE / RETRY STATE
*
=====

=====
*/
bool pausedByObstacle = false;
bool waitingForRetry = false;
unsigned long obstacleRetryTime = 0;

/*
=====

=====
* SECURITY STATE
*
=====

=====
*/
bool securityAtNinetyActive = false;

```

```

/*
=====

=====
* GREEN LED AUTHORIZATION
*
=====

=====
*/
bool zeroCommandFromWebsite = false;

/*
=====

=====
* MPU: LOW-LEVEL I2C WRITE
*
=====

=====
*/
void writeMPU(byte reg, byte value)
{
    Wire.beginTransaction(MPU_ADDR);
    Wire.write(reg);
    Wire.write(value);
    Wire.endTransmission();
}

/*
=====

=====
* MPU: READ ACCELEROMETER + COMPUTE POSITION ANGLE
*
=====

=====
* Reads the 14-byte accel/temp/gyro block, extracts AX/AZ, applies a
simple
* low-pass filter, and recomputes positionAngle from the filtered
values.
* Returns false on any I2C communication failure.
*/
bool readMPU()
{

```

```

Wire.beginTransaction(MPU_ADDR);
Wire.write(MPU_ACCEL_XOUT);

if (Wire.endTransmission(false) != 0)
{
    return false;
}

int received = Wire.requestFrom(MPU_ADDR, 14, true);

if (received != 14)
{
    return false;
}

int16_t rawAx = (Wire.read() << 8) | Wire.read();
int16_t rawAy = (Wire.read() << 8) | Wire.read();
int16_t rawAz = (Wire.read() << 8) | Wire.read();

// Skip temperature (2 bytes)
Wire.read();
Wire.read();

// Skip gyro X (2 bytes)
Wire.read();
Wire.read();

// Skip gyro Y (2 bytes)
Wire.read();
Wire.read();

// Skip gyro Z (2 bytes)
Wire.read();
Wire.read();

ax = rawAx / 16384.0f;
az = rawAz / 16384.0f;

// Low-pass filtering (70% history / 30% new sample)
filteredAx = (filteredAx * 0.70f) + (ax * 0.30f);
filteredAz = (filteredAz * 0.70f) + (az * 0.30f);

// Calculate orientation. This gives a monotonic position through

```

```

// the ParkBnB movement range.
positionAngle = atan2(filteredAx, filteredAz) * 180.0f / PI;

return true;
}

/*
=====
=====
* MPU: INITIALIZATION
*
=====
=====
* Averages 30 samples to establish a stable starting filtered AX/AZ
and
* initial positionAngle. Returns false if any sample read fails.
*/
bool initializeMPU()
{
    float totalAx = 0.0f;
    float totalAz = 0.0f;
    const int samples = 30;

    for (int i = 0; i < samples; i++)
    {
        if (!readMPU())
        {
            return false;
        }

        totalAx += ax;
        totalAz += az;

        delay(20);
    }

    filteredAx = totalAx / samples;
    filteredAz = totalAz / samples;

    positionAngle = atan2(filteredAx, filteredAz) * 180.0f / PI;

    return true;
}

```

```

}

/*
=====
=====
* MOTOR: PWM SETUP
*
=====
=====
*/
void setupMotorPWM()
{
    ledcAttach(RPWM_PIN, PWM_FREQ, PWM_RESOLUTION);
    ledcAttach(LPWM_PIN, PWM_FREQ, PWM_RESOLUTION);
}

/*
=====
=====
* MOTOR: ENABLE DRIVER
*
=====
=====
*/
void enableMotorDriver()
{
    digitalWrite(REN_PIN, HIGH);
    digitalWrite(LEN_PIN, HIGH);
}

/*
=====
=====
* MOTOR: EMERGENCY STOP
*
=====
=====
* Immediately kills PWM output and disables the driver enable pins.
* Used for obstacle detection, timeouts, and MPU failures.
*/

```

```

void emergencyMotorStop()
{
    ledcWrite(RPWM_PIN, 0);
    ledcWrite(LPWM_PIN, 0);

    digitalWrite(REN_PIN, LOW);
    digitalWrite(LEN_PIN, LOW);

    motorState = STOPPED;
    noMovementCount = 0;

    Serial.println("!!! MOTOR EMERGENCY STOP !!!");
}

/*
=====
=====
* MOTOR: NORMAL STOP
*
=====
=====
* Used when a target position has been reached successfully. Clears
the
* commanded target and obstacle/retry flags.
*/
void normalMotorStop()
{
    ledcWrite(RPWM_PIN, 0);
    ledcWrite(LPWM_PIN, 0);

    motorState = STOPPED;
    commandedTarget = STOPPED;

    pausedByObstacle = false;
    waitingForRetry = false;

    noMovementCount = 0;

    Serial.println("MOTOR STOPPED");
}

```

```

/*
=====
=====
* MOTOR: MOVE TO ZERO
*
=====
=====
* Drives via LPWM. positionAngle decreases toward approximately
-72.7°.
*/
void moveMotorToZero()
{
    enableMotorDriver();

    ledcWrite(RPWM_PIN, 0);
    ledcWrite(LPWM_PIN, MOTOR_SPEED);

    motorState = MOVING_TO_ZERO;

    Serial.println("MOTOR MOVING TO 0");
}

/*
=====
=====
* MOTOR: MOVE TO NINETY
*
=====
=====
* Drives via RPWM. positionAngle increases toward approximately +6.4°.
*/
void moveMotorToNinety()
{
    enableMotorDriver();

    ledcWrite(RPWM_PIN, MOTOR_SPEED);
    ledcWrite(LPWM_PIN, 0);

    motorState = MOVING_TO_NINETY;

    Serial.println("MOTOR MOVING TO 90");
}

```

```

/*
=====
=====
* MOTOR: RETURN FROM ABOVE 90
*
=====
=====
* When the physical position is above 90°, positionAngle is greater
than the
* 90° reference. The motor must therefore run in the LPWM direction to
bring
* positionAngle back down toward +6.4°.
*/
void moveMotorDownFromOverNinety()
{
    enableMotorDriver();

    ledcWrite(RPWM_PIN, 0);
    ledcWrite(LPWM_PIN, MOTOR_SPEED);

    motorState = MOVING_DOWN_FROM_OVER_NINETY;

    Serial.println("MOTOR REVERSE: RETURNING FROM ABOVE 90");
}

/*
=====
=====
* BUZZER CONTROL
*
=====
=====
*/
void startBuzzer()
{
    digitalWrite(BUZZER_PIN, HIGH);

    buzzerActive = true;
    buzzerStartTime = millis();
}

```

```

    Serial.println("!!! BUZZER ON !!!");
}

void stopBuzzer()
{
    digitalWrite(BUZZER_PIN, LOW);

    buzzerActive = false;

    Serial.println("BUZZER OFF");
}

/*
=====
=====
* STALL DETECTOR RESET
*
=====
=====
* Called whenever a new movement begins, to give the stall detector a
fresh
* reference angle/time.
*/
void resetStallDetector()
{
    lastStallCheckAngle = positionAngle;
    lastStallCheckTime  = millis();
    noMovementCount = 0;
}

/*
=====
=====
* OBSTACLE DETECTED
*
=====
=====
* Stops the motor immediately and starts the buzzer alarm. The
obstacle
* recovery routine will retry the original movement once the buzzer
cycle

```

```

    * completes.
    */
void obstacleDetected()
{
    if (pausedByObstacle)
    {
        return;
    }

    Serial.println();
    Serial.println("#####");
    Serial.println("OBSTACLE DETECTED");
    Serial.println("STOPPING MOTOR");
    Serial.println("BARRIER WILL RETRY AFTER BUZZER");
    Serial.println("#####");

    emergencyMotorStop();

    pausedByObstacle = true;
    waitingForRetry = false;

    startBuzzer();
}

/*
=====
=====
* OBSTACLE RECOVERY
*
=====
=====
* State flow:
* 1. Buzzer sounds for BUZZER_DURATION.
* 2. Once buzzer stops, wait OBSTACLE_RETRY_DELAY.
* 3. Retry whichever movement (commandedTarget) was in progress.
*/
void updateObstacleRecovery()
{
    if (buzzerActive)
    {
        if (millis() - buzzerStartTime >= BUZZER_DURATION)
        {

```

```
    stopBuzzer();

    waitingForRetry = true;
    obstacleRetryTime = millis();

    Serial.println("BUZZER CYCLE COMPLETE");
}

return;
}

if (!pausedByObstacle)
{
    return;
}

if (!waitingForRetry)
{
    return;
}

if (millis() - obstacleRetryTime < OBSTACLE_RETRY_DELAY)
{
    return;
}

waitingForRetry = false;

Serial.println("RETRYING ORIGINAL MOVEMENT");

resetStallDetector();

pausedByObstacle = false;

motorStartTime = millis();

if (commandedTarget == MOVING_TO_ZERO)
{
    moveMotorToZero();
}
else if (commandedTarget == MOVING_TO_NINETY)
{
    moveMotorToNinety();
}
```

```

    }
    else if (commandedTarget == MOVING_DOWN_FROM_OVER_NINETY)
    {
        moveMotorDownFromOverNinety();
    }
    else
    {
        pausedByObstacle = false;
        waitingForRetry = false;
    }
}

/*
=====
=====
* SECURITY CHECK: MOVED BELOW 90
*
=====
=====
* If the tyre drifts below 90° while locked (positionAngle <
NINETY_LOWER_LIMIT),
* trigger an alarm and command a return move (RPWM /
moveMotorToNinety()).
*/
void checkNinetySecurity()
{
    if (!securityAtNinetyActive)
    {
        return;
    }

    if (motorState != STOPPED)
    {
        return;
    }

    if (pausedByObstacle)
    {
        return;
    }

    if (buzzerActive)

```

```

    {
        return;
    }

    if (positionAngle < NINETY_LOWER_LIMIT)
    {
        Serial.println();
        Serial.println("SECURITY: MOVED BELOW 90");

        commandedTarget = MOVING_TO_NINETY;

        pausedByObstacle = true;
        waitingForRetry = false;

        startBuzzer();
    }
}

/*
=====
=====
* SECURITY CHECK: MOVED ABOVE 90
*
=====
=====
* If the tyre moves above physical 90° (positionAngle >
NINETY_UPPER_LIMIT),
* trigger an alarm and command a reverse move (LPWM / reverse
direction).
*/
void checkOverNinety()
{
    if (!securityAtNinetyActive)
    {
        return;
    }

    if (motorState != STOPPED)
    {
        return;
    }
}

```

```

    if (pausedByObstacle)
    {
        return;
    }

    if (buzzerActive)
    {
        return;
    }

    if (positionAngle > NINETY_UPPER_LIMIT)
    {
        Serial.println();
        Serial.println("OVER 90 DETECTED");
        Serial.println("REVERSE DIRECTION REQUIRED");

        commandedTarget = MOVING_DOWN_FROM_OVER_NINETY;

        pausedByObstacle = true;
        waitingForRetry = false;

        startBuzzer();
    }
}

/*
=====
=====
* LED UPDATE
*
=====
=====
* Green ("zero") LED lights only when the barrier is at/near 0° AND
the
* open command actually came from the website
(zeroCommandFromWebsite).
* Otherwise the "ninety" LED is lit.
*/
void updatePositionLEDs()
{
    if (positionAngle > ZERO_UPPER_LIMIT)
    {

```

```

    zeroCommandFromWebsite = false;
}

if (positionAngle <= ZERO_UPPER_LIMIT && zeroCommandFromWebsite)
{
    digitalWrite(LED_ZERO_PIN, HIGH);
    digitalWrite(LED_NINETY_PIN, LOW);
}
else
{
    digitalWrite(LED_NINETY_PIN, HIGH);
    digitalWrite(LED_ZERO_PIN, LOW);
}
}

/*
=====
=====
* MOVEMENT CHECK: TOWARD ZERO
*
=====
=====
* Detects arrival at 0° and performs stall detection while en route.
*/
void checkMovementToZero()
{
    if (positionAngle <= ZERO_UPPER_LIMIT)
    {
        Serial.println("0 DEGREE REACHED");

        normalMotorStop();

        securityAtNinetyActive = false;
        zeroCommandFromWebsite = true;

        return;
    }

    if (millis() - lastStallCheckTime < STALL_CHECK_INTERVAL)
    {
        return;
    }
}

```

```

float movement = lastStallCheckAngle - positionAngle;

if (movement < MIN_MOVEMENT)
{
    noMovementCount++;
}
else
{
    noMovementCount = 0;
}

lastStallCheckAngle = positionAngle;
lastStallCheckTime = millis();

if (noMovementCount >= MAX_NO_MOVEMENT_COUNT)
{
    obstacleDetected();
}
}

/*
=====
=====
* MOVEMENT CHECK: TOWARD NINETY
*
=====
=====
* Angle should increase toward NINETY_ANGLE_REFERENCE.
*/
void checkMovementToNinety()
{
    if (positionAngle >= NINETY_ANGLE_REFERENCE)
    {
        Serial.println("90 DEGREE REACHED");

        normalMotorStop();

        securityAtNinetyActive = true;

        return;
    }
}

```

```

    if (millis() - lastStallCheckTime < STALL_CHECK_INTERVAL)
    {
        return;
    }

    float movement = positionAngle - lastStallCheckAngle;

    if (movement < MIN_MOVEMENT)
    {
        noMovementCount++;
    }
    else
    {
        noMovementCount = 0;
    }

    lastStallCheckAngle = positionAngle;
    lastStallCheckTime = millis();

    if (noMovementCount >= MAX_NO_MOVEMENT_COUNT)
    {
        obstacleDetected();
    }
}

/*
=====
=====
* MOVEMENT CHECK: RETURN FROM ABOVE 90
*
=====
=====
* Angle should DECREASE back toward NINETY_ANGLE_REFERENCE (LPWM
reverse
* direction).
*/
void checkMovementDownFromOverNinety()
{
    if (positionAngle <= NINETY_ANGLE_REFERENCE)
    {
        Serial.println("RETURNED TO 90 FROM ABOVE");
    }
}

```

```

    normalMotorStop();

    securityAtNinetyActive = true;

    return;
}

if (millis() - lastStallCheckTime < STALL_CHECK_INTERVAL)
{
    return;
}

float movement = lastStallCheckAngle - positionAngle;

if (movement < MIN_MOVEMENT)
{
    noMovementCount++;
}
else
{
    noMovementCount = 0;
}

lastStallCheckAngle = positionAngle;
lastStallCheckTime  = millis();

if (noMovementCount >= MAX_NO_MOVEMENT_COUNT)
{
    obstacleDetected();
}
}

/*
=====
=====
*  COMMAND: MOVE TO ZERO (OPEN)
*
=====
=====
*/
void commandZero()

```

```

{
    Serial.println();
    Serial.println("COMMAND: MOVE TO 0 DEGREE");

    securityAtNinetyActive = false;

    pausedByObstacle = false;
    waitingForRetry = false;

    if (buzzerActive)
    {
        stopBuzzer();
    }

    if (!readMPU())
    {
        Serial.println("MPU ERROR");
        return;
    }

    if (positionAngle <= ZERO_UPPER_LIMIT)
    {
        Serial.println("ALREADY AT 0 DEGREE");

        normalMotorStop();

        zeroCommandFromWebsite = true;

        return;
    }

    commandedTarget = MOVING_TO_ZERO;
    motorStartTime = millis();

    resetStallDetector();

    moveMotorToZero();
}

```

```

/*

```

```

=====

```

```

=====

```

```

* COMMAND: MOVE TO NINETY (CLOSE)
*
=====
=====
* If current position is below 90°: drive RPWM (forward).
* If current position is above 90°: drive LPWM (reverse).
*/
void commandNinety()
{
    Serial.println();
    Serial.println("COMMAND: MOVE TO 90 DEGREE");

    pausedByObstacle = false;
    waitingForRetry = false;

    zeroCommandFromWebsite = false;

    if (buzzerActive)
    {
        stopBuzzer();
    }

    if (!readMPU())
    {
        Serial.println("MPU ERROR");
        return;
    }

    // Already inside the 90-degree tolerance window.
    if (positionAngle >= NINETY_LOWER_LIMIT && positionAngle <=
NINETY_UPPER_LIMIT)
    {
        Serial.println("ALREADY AT 90 DEGREE");

        normalMotorStop();

        securityAtNinetyActive = true;

        return;
    }

    motorStartTime = millis();

```

```

    resetStallDetector();

    // BELOW 90
    if (positionAngle < NINETY_LOWER_LIMIT)
    {
        commandedTarget = MOVING_TO_NINETY;

        moveMotorToNinety();

        return;
    }

    // ABOVE 90
    if (positionAngle > NINETY_UPPER_LIMIT)
    {
        commandedTarget = MOVING_DOWN_FROM_OVER_NINETY;

        moveMotorDownFromOverNinety();

        return;
    }
}

/*
=====
=====
* WEB: STATUS STRING
*
=====
=====
*/
String getStatusString()
{
    if (pausedByObstacle)
    {
        return "OBSTACLE / SECURITY ALARM";
    }

    if (motorState == STOPPED)
    {
        if (securityAtNinetyActive)
        {

```

```

        return "LOCKED AT 90 - MONITORING";
    }

    return "STOPPED";
}

if (motorState == MOVING_TO_ZERO)
{
    return "MOVING TO 0";
}

if (motorState == MOVING_TO_NINETY)
{
    return "MOVING TO 90";
}

if (motorState == MOVING_DOWN_FROM_OVER_NINETY)
{
    return "RETURNING FROM ABOVE 90";
}

return "UNKNOWN";
}

/*
=====
=====
* WEB: ROOT PAGE
*
=====
=====
*/
void handleRoot()
{
    String status = getStatusString();

    String page = R"HTML(
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>ParkBnB</title>

```

```

<style>
body
{
    font-family: Arial;
    text-align: center;
    background: #f2f2f2;
}
button
{
    width: 240px;
    height: 70px;
    margin: 10px;
    font-size: 24px;
}
</style>
</head>
<body>
<h1>ParkBnB Control</h1>
<h2>STATUS:
)HTML";

    page += status;

    page += R"HTML(
</h2>
<p>Position Angle:
)HTML";

    page += String(positionAngle, 2);

    page += R"HTML(
degrees</p>
<p>AX:
)HTML";

    page += String(filteredAx, 3);

    page += R"HTML(
</p>
<p>AZ:
)HTML";

    page += String(filteredAz, 3);

```

```

    page += R"HTML(
</p>
<a href="/zero">
<button>OPEN 0&deg;</button>
</a>
<br>
<a href="/ninety">
<button>CLOSE 90&deg;</button>
</a>
<br>
<a href="/stop">
<button>STOP</button>
</a>
</body>
</html>
)HTML";

    server.send(200, "text/html", page);
}

```

```

/*
=====

=====
* WEB: ROUTE HANDLERS
*
=====

=====
*/
void redirectToRoot()
{
    server.setHeader("Location", "/");
    server.send(303);
}

void handleZero()
{
    commandZero();
    redirectToRoot();
}

void handleNinety()

```

```

{
    commandNinety();
    redirectToRoot();
}

void handleStop()
{
    emergencyMotorStop();

    commandedTarget = STOPPED;

    pausedByObstacle = false;
    waitingForRetry = false;

    securityAtNinetyActive = false;
    zeroCommandFromWebsite = false;

    if (buzzerActive)
    {
        stopBuzzer();
    }

    redirectToRoot();
}

void setupWebRoutes()
{
    server.on("/", handleRoot);
    server.on("/zero", handleZero);
    server.on("/ninety", handleNinety);
    server.on("/stop", handleStop);

    server.begin();

    Serial.println("WEB SERVER STARTED");
}

/*
=====
=====
* OUTPUT SETUP

```

```

*
=====

=====
*/
void setupOutputs()
{
    pinMode(BUZZER_PIN, OUTPUT);
    digitalWrite(BUZZER_PIN, LOW);

    pinMode(REN_PIN, OUTPUT);
    pinMode(LEN_PIN, OUTPUT);
    digitalWrite(REN_PIN, LOW);
    digitalWrite(LEN_PIN, LOW);

    pinMode(LED_NINETY_PIN, OUTPUT);
    pinMode(LED_ZERO_PIN, OUTPUT);
    digitalWrite(LED_NINETY_PIN, LOW);
    digitalWrite(LED_ZERO_PIN, LOW);

    setupMotorPWM();

    emergencyMotorStop();
}

/*
=====

=====
*/
void setupMPU()
{
    Wire.beginTransmission(MPU_ADDR);

    if (Wire.endTransmission() != 0)
    {
        Serial.println("MPU6050 NOT FOUND");

        while (true)
        {

```

```

        delay(1000);
    }
}

Serial.println("MPU6050 FOUND");

writeMPU(MPU_PWR_MGMT_1, 0x00);

delay(100);

if (!initializeMPU())
{
    Serial.println("MPU INITIALIZATION FAILED");

    while (true)
    {
        delay(1000);
    }
}

Serial.print("INITIAL POSITION ANGLE: ");
Serial.println(positionAngle, 2);
}

/*
=====
=====
* WI-FI SETUP
*
=====
=====
*/
void setupWiFi()
{
    WiFi.begin(ssid, password);

    Serial.print("CONNECTING TO WIFI");

    while (WiFi.status() != WL_CONNECTED)
    {
        delay(500);
        Serial.print(".");
    }
}

```

```

    }

    Serial.println();
    Serial.println("WIFI CONNECTED");
    Serial.print("IP ADDRESS: ");
    Serial.println(WiFi.localIP());
}

/*
=====
=====
*   STARTUP POSITION CHECK
*
=====
=====
*   On boot, detects whether the tyre is below, at, or above physical
90°
*   and moves it into the locked 90° position accordingly.
*/
void setupStartupPosition()
{
    Serial.println();
    Serial.println("STARTUP: CHECKING TYRE POSITION");

    motorStartTime = millis();

    resetStallDetector();

    // Already near physical 90.
    if (positionAngle >= NINETY_LOWER_LIMIT && positionAngle <=
NINETY_UPPER_LIMIT)
    {
        Serial.println("STARTUP: ALREADY AT 90 DEGREE");

        normalMotorStop();

        securityAtNinetyActive = true;

        return;
    }

    // Below physical 90.

```

```

if (positionAngle < NINETY_LOWER_LIMIT)
{
    Serial.println("STARTUP: BELOW 90 -> MOVING FORWARD");

    commandedTarget = MOVING_TO_NINETY;

    moveMotorToNinety();

    return;
}

// Above physical 90.
if (positionAngle > NINETY_UPPER_LIMIT)
{
    Serial.println("STARTUP: ABOVE 90 -> REVERSING");

    commandedTarget = MOVING_DOWN_FROM_OVER_NINETY;

    moveMotorDownFromOverNinety();

    return;
}
}

/*
=====
=====
* MPU READ FAILURE HANDLER
*
=====
=====
*/
void handleMpuReadFailure()
{
    if (motorState != STOPPED)
    {
        Serial.println("MPU ERROR WHILE MOTOR MOVING");

        emergencyMotorStop();

        commandedTarget = STOPPED;
        securityAtNinetyActive = false;
    }
}

```

```

        startBuzzer();
    }
}

/*
=====
=====
* MOVEMENT UPDATE DISPATCHER
*
=====
=====
* Routes to the correct movement-check function based on motorState,
and
* enforces the overall motor timeout.
*/
void updateMovementChecks()
{
    if (pausedByObstacle)
    {
        return;
    }

    if (motorState == MOVING_TO_ZERO)
    {
        checkMovementToZero();
    }
    else if (motorState == MOVING_TO_NINETY)
    {
        checkMovementToNinety();
    }
    else if (motorState == MOVING_DOWN_FROM_OVER_NINETY)
    {
        checkMovementDownFromOverNinety();
    }

    if (motorState != STOPPED)
    {
        if (millis() - motorStartTime >= MOTOR_TIMEOUT)
        {
            Serial.println("MOTOR TIMEOUT");

```

```

        obstacleDetected();
    }
}

/*
=====
=====
* SERIAL STATUS PRINTOUT
*
=====
=====
* Throttled to print at most once every 300ms.
*/
void printSerialStatus()
{
    static unsigned long lastPrint = 0;

    if (millis() - lastPrint < 300)
    {
        return;
    }

    lastPrint = millis();

    Serial.print("ANGLE: ");
    Serial.print(positionAngle, 2);
    Serial.print(" | AX: ");
    Serial.print(filteredAx, 3);
    Serial.print(" | AZ: ");
    Serial.print(filteredAz, 3);
    Serial.print(" | STATE: ");

    if (pausedByObstacle)
    {
        Serial.println("ALARM / WAIT");
    }
    else if (motorState == STOPPED)
    {
        if (securityAtNinetyActive)
        {
            Serial.println("90 MONITORING");
        }
    }
}

```

```

    }
    else
    {
        Serial.println("STOPPED");
    }
}
else if (motorState == MOVING_TO_ZERO)
{
    Serial.println("TO 0");
}
else if (motorState == MOVING_TO_NINETY)
{
    Serial.println("TO 90 FORWARD");
}
else if (motorState == MOVING_DOWN_FROM_OVER_NINETY)
{
    Serial.println("TO 90 REVERSE");
}
}

/*
=====
=====
* FIREBASE: INITIALIZE PATHS
*
=====
=====
* Builds the three Firebase Realtime Database REST endpoint URLs used
by
* this device.
*/
void initializeFirebasePaths()
{
    firebaseDeviceURL = String(DATABASE_URL) + "/barriers/" +
String(BARRIER_ID) + "/device.json";
    firebaseLiveDataURL = String(DATABASE_URL) + "/barriers/" +
String(BARRIER_ID) + "/liveData.json";
    firebaseCommandURL = String(DATABASE_URL) + "/barriers/" +
String(BARRIER_ID) + "/command/action.json";

    Serial.println();
    Serial.println("=====");
}

```

```

Serial.println("FIREBASE PATHS INITIALIZED");
Serial.println("=====");

Serial.print("DEVICE URL: ");
Serial.println(firebaseDeviceURL);

Serial.print("LIVE DATA URL: ");
Serial.println(firebaseLiveDataURL);

Serial.print("COMMAND URL: ");
Serial.println(firebaseCommandURL);
}

/*
=====
=====
* FIREBASE: PATCH REQUEST
*
=====
=====
* Sends an HTTP PATCH with a JSON payload to the given Firebase URL.
* Returns true on HTTP 2xx success.
*
* NOTE: client.setInsecure() disables TLS certificate validation.
* This is marked "TESTING ONLY" in the original code and should be
* replaced with proper certificate validation before production use.
*/
bool firebasePatch(const String &url, const String &payload)
{
    // Check Wi-Fi connection
    if (WiFi.status() != WL_CONNECTED)
    {
        Serial.println("FIREBASE ERROR: WIFI NOT CONNECTED");
        return false;
    }

    // Create secure client
    WiFiClientSecure client;

    // TESTING ONLY: disable SSL certificate validation
    client.setInsecure();

```

```

// Create HTTPS client
HTTPClient https;

// Start HTTPS connection
if (!https.begin(client, url))
{
    Serial.println("FIREBASE ERROR: HTTPS CONNECTION FAILED");
    return false;
}

// JSON content type
https.addHeader("Content-Type", "application/json");

// Send PATCH request
int httpCode = https.sendRequest("PATCH", payload);

Serial.print("FIREBASE HTTP CODE: ");
Serial.println(httpCode);

bool success = false;

if (httpCode >= 200 && httpCode < 300)
{
    String response = https.getString();

    Serial.print("FIREBASE RESPONSE: ");
    Serial.println(response);

    success = true;
}
else
{
    Serial.print("FIREBASE ERROR: ");
    Serial.println(https.errorToString(httpCode));
}

https.end();

return success;
}

```

```

/*
=====
=====
* FIREBASE: GET REQUEST
*
=====
=====
* Fetches the raw response body from the given Firebase URL into
* `response`. Returns true on HTTP 2xx success.
*/
bool firebaseGet(const String &url, String &response)
{
    if (WiFi.status() != WL_CONNECTED)
    {
        Serial.println("FIREBASE GET ERROR: WIFI NOT CONNECTED");
        return false;
    }

    WiFiClientSecure client;
    client.setInsecure();

    HTTPClient https;

    if (!https.begin(client, url))
    {
        Serial.println("FIREBASE GET ERROR: HTTPS CONNECTION FAILED");
        return false;
    }

    int httpCode = https.GET();

    Serial.print("FIREBASE GET HTTP CODE: ");
    Serial.println(httpCode);

    bool success = false;

    if (httpCode >= 200 && httpCode < 300)
    {
        response = https.getString();
        success = true;
    }
    else
    {

```

```

        Serial.print("FIREBASE GET ERROR: ");
        Serial.println(https.errorToString(httpCode));
    }

    https.end();

    return success;
}

/*
=====
=====
* FIREBASE: CHECK REMOTE COMMAND
*
=====
=====
* Polls the Firebase "command/action" node and executes OPEN/CLOSE
commands.
* Only runs when the motor is idle and not mid-obstacle/buzzer
sequence,
* throttled to FIREBASE_COMMAND_CHECK_INTERVAL.
*/
void checkFirebaseCommand()
{
    if (WiFi.status() != WL_CONNECTED)
    {
        return;
    }

    if (motorState != STOPPED)
    {
        return;
    }

    if (pausedByObstacle)
    {
        return;
    }

    if (waitingForRetry)
    {
        return;
    }

```

```

    }

    if (buzzerActive)
    {
        return;
    }

    if (millis() - lastFirebaseCommandCheck <
FIREBASE_COMMAND_CHECK_INTERVAL)
    {
        return;
    }

    lastFirebaseCommandCheck = millis();

    String response;

    Serial.println();
    Serial.println("=====");
    Serial.println("CHECKING FIREBASE COMMAND");
    Serial.println("=====");

    if (!firebaseGet(firebaseCommandURL, response))
    {
        return;
    }

    response.trim();

    Serial.print("FIREBASE COMMAND RECEIVED: ");
    Serial.println(response);

    // ---- Firebase command execution ----

    // OPEN command
    if (response == "\"OPEN\"")
    {
        Serial.println("FIREBASE EXECUTING: OPEN -> 0 DEGREE");

        commandZero();

        return;
    }

```

```

// CLOSE command
if (response == "\"CLOSE\"")
{
    Serial.println("FIREBASE EXECUTING: CLOSE -> 90 DEGREE");

    commandNinety();

    return;
}
}

/*
=====
=====
* FIREBASE: SEND DEVICE DATA
*
=====
=====
* Publishes device identity/heartbeat information (deviceId, status,
* Wi-Fi connectivity, uptime) to the /device node.
*/
void sendFirebaseDeviceData()
{
    StaticJsonDocument<256> json;

    json["deviceId"]      = BARRIER_ID;
    json["status"]        = "online";
    json["wifiConnected"] = (WiFi.status() == WL_CONNECTED);
    json["uptimeMs"]      = millis();

    String payload;
    serializeJson(json, payload);

    Serial.println();
    Serial.println("=====");
    Serial.println("SENDING FIREBASE DEVICE DATA");
    Serial.println("=====");
    Serial.println(payload);

    firebasePatch(firebaseDeviceURL, payload);
}

```

```

/*
=====
=====
* FIREBASE: SEND LIVE DATA
*
=====
=====
* Publishes the current barrier position ("0 DEGREE" / "90 DEGREE" /
* "MOVING"), raw position angle, and buzzer state to the /liveData
node.
*/
void sendFirebaseLiveData()
{
    StaticJsonDocument<256> json;

    String barrierPosition;

    // Barrier at 0 degrees
    if (positionAngle <= ZERO_UPPER_LIMIT)
    {
        barrierPosition = "0 DEGREE";
    }
    // Barrier at 90 degrees
    else if (positionAngle >= NINETY_LOWER_LIMIT && positionAngle <=
NINETY_UPPER_LIMIT)
    {
        barrierPosition = "90 DEGREE";
    }
    // Barrier between 0 and 90 degrees
    else
    {
        barrierPosition = "MOVING";
    }

    json["barrierPosition"] = barrierPosition;
    json["positionAngle"]   = positionAngle;
    json["buzzerActive"]    = buzzerActive;

    String payload;
    serializeJson(json, payload);
}

```

```

    Serial.println();
    Serial.println("=====");
    Serial.println("SENDING FIREBASE LIVE DATA");
    Serial.println("=====");
    Serial.println(payload);

    firebasePatch(firebaseLiveDataURL, payload);
}

/*
=====
=====
* SETUP
*
=====
=====
*/
void setup()
{
    Serial.begin(115200);

    Wire.begin(21, 22);

    setupOutputs();
    setupMPU();
    setupWiFi();
    setupWebRoutes();
    setupStartupPosition();

    updatePositionLEDs();

    // ---- Firebase path initialization ----
    initializeFirebasePaths();

    // ---- Send initial Firebase data ----
    if (WiFi.status() == WL_CONNECTED)
    {
        sendFirebaseDeviceData(); // device information
        sendFirebaseLiveData(); // live ParkBnB status
    }
    else
    {

```

```

        Serial.println("FIREBASE STARTUP SEND SKIPPED: WIFI NOT
CONNECTED");
    }
}

/*
=====
=====
* MAIN LOOP
*
=====
=====
*/
void loop()
{
    server.handleClient();

    bool mpuOK = readMPU();

    if (!mpuOK)
    {
        handleMpuReadFailure();
        return;
    }

    updateObstacleRecovery();
    checkNinetySecurity();
    checkOverNinety();
    updateMovementChecks();
    updatePositionLEDs();
    printSerialStatus();

    // ---- Periodic Firebase live-data update ----
    if (WiFi.status() == WL_CONNECTED &&
        motorState == STOPPED &&
        millis() - lastFirebaseLiveUpdate >=
FIREBASE_LIVE_UPDATE_INTERVAL)
    {
        lastFirebaseLiveUpdate = millis();

        sendFirebaseLiveData();
    }
}

```

```
// ---- Firebase command check ----  
if (WiFi.status() == WL_CONNECTED && motorState == STOPPED)  
{  
    checkFirebaseCommand();  
}  
}
```